Security Analysis
Aplicația StudySync

Echipa CFC

1 Arhitectura de Securitate Curentă

Sistemul actual de securitate este construit pe framework-ul **Spring Security** și utilizează următoarele componente:

- **Autentificare Stateless:** Se bazează pe **JWT** (JSON Web Tokens) semnate cu algoritmul `HS256`. Token-urile includ claims pentru `userId`, `role` și `email`, având o durată de viață de 24 de ore.
- **Protecția Datelor:** Parolele utilizatorilor sunt procesate folosind funcția de hashing **BCrypt**.
- **Configurare Pipeline:** Accesul este gestionat printr-un `JwtAuthFilter` custom, inserat înaintea filtrului standard de autentificare.
- **Comunicare Real-time:** WebSocket-urile sunt implementate prin protocolul STOMP peste SockJS, cu endpoint-uri segregate pe topic-uri de tip `spaceId`.

2 Riscuri de Securitate Identificate

2.1 Autentificare și Managementul Sesiunilor

- **Gestiunea Secretelor:** Cheia `JWT_SECRET` este stocată static în fișierul `.env` fără rotație, riscând compromiterea întregului sistem de token-uri.
- **Persistența Token-urilor:** Lipsa mecanismului de *refresh tokens* forțează utilizarea unor sesiuni lungi care nu pot fi invalidate în caz de furt.
- **Lipsa Revocării:** Deconectarea (*logout*) nu invalidează token-ul pe partea de server, acesta rămânând utilizabil până la expirare.

2.2 Validarea Datelor și Input-uri

- **Absența Validării DTO-urilor:** Lipsa adnotărilor Bean Validation (`@Valid`, `@NotNull`) permite acceptarea parolelor slabe sau a payload-urilor malformate.
- **Vulnerabilitate NoSQL Injection:** Intrarea de date nesanitizată în MongoDB poate permite manipularea interogărilor prin operatori JSON.
- **Securitatea Fișierelor:** Limitele de upload excesive (50MB) și lipsa verificării semnăturii fișierelor permit atacuri de tip DoS sau execuție de cod la distanță.

2.3 Canale de Comunicare (WebSocket)

- **Acces Public:** Endpoint-ul `/ws/**` este configurat cu `permitAll()`, permițând oricărui utilizator neautentificat să inițieze conexiuni.
- **Lipsa Autorizării pe Topic:** Nu se verifică drepturile de membru la subscrierea pe un `spaceId`, permițând scurgerea de date între utilizatori.
- **CORS Permisiv:** Utilizarea `setAllowedOriginPatterns("*")` expune aplicația la atacuri de tip CSRF asupra conexiunilor WebSocket.

2.4 Autorizarea Resurselor

- **Securitate Nivel Metodă:** Absența `@PreAuthorize` face ca protecția resurselor să fie dependentă de verificări manuale, predispuse la erori umane.
- **Vulnerabilități IDOR:** Verificările de apartenență (*ownership*) sunt incomplete, permițând potențiala modificare a resurselor altor utilizatori prin manipularea ID-urilor.

3 Strategii de Remediere

3.1 Securizarea WebSocket

Implementarea unui `ChannelInterceptor` care să valideze token-ul JWT la comanda `CONNECT` și să verifice drepturile de acces la topic în momentul comenzii `SUBSCRIBE`.

3.2 Validare și Igienizare

Activarea `hibernate-validator` în controllere și implementarea unui sistem de sanitizare pentru caracterele speciale pentru a preveni atacurile XSS și NoSQL Injection.

3.3 Gestiunea Avansată a Token-urilor

Introducerea perechii *Access Token* (scurt) / *Refresh Token* (lung) și stocarea acestora într-o bază de date pentru a permite revocarea instantanee a sesiunilor.

3.4 Protecție și Monitorizare

Configurarea *Rate Limiting*-ului pentru endpoint-urile sensibile (Login/AI) și implementarea unui sistem de *Audit Logging* pentru trasabilitatea acțiunilor critice.